5\. Document Chunking Strategies

Document chunking is the process of dividing large documents into smaller segments before generating embeddings. Chunking is important because embedding models have token limits, and retrieving an entire document may introduce irrelevant information into the language model's context. Fixed-size chunking divides documents according to a predetermined number of tokens or characters. Recursive chunking attempts to preserve natural document boundaries such as paragraphs, sentences, and headings. Semantic chunking uses changes in meaning to determine where one segment should end and another should begin. Overlapping chunks share a portion of their content to preserve context across boundaries. For example, a system using 500-token chunks with a 50-token overlap will retain some information from the previous chunk in each subsequent segment.

6\. Message Queues and Background Processing

Message queues allow different components of an application to communicate asynchronously. A producer publishes messages to a queue, while one or more consumers retrieve and process those messages. This architecture is useful for background jobs such as sending emails, processing uploaded documents, generating embeddings, and executing scheduled tasks. Systems such as RabbitMQ, Apache Kafka, and Redis Streams provide different approaches to message delivery and processing. A major advantage of message queues is that they decouple application components, allowing producers and consumers to operate independently. However, distributed message processing introduces challenges involving duplicate messages, failed jobs, message ordering, and retry policies. Idempotent consumers help prevent duplicate processing from producing unintended side effects.